Terrafrom_2 Task

🔑 Why Terraform is important

1. **Infrastructure as Code (IaC)**
   Write infrastructure in .tf files → version-controlled, repeatable, and auditable.
2. **Automation & Speed**
   Create servers, networks, databases, and services in minutes with init → plan → apply.
3. **Consistency & No Human Error**
   Same code = same infrastructure across dev, test, and prod.
4. **Multi-Cloud Support**
   Works with AWS, Azure, GCP, and on-prem from a **single tool**.
5. **State Management**
   Tracks real infrastructure using a **state file**, enabling safe updates and drift detection.
6. **Idempotency**
   Running Terraform multiple times won't break infra — it only changes what's needed.
7. **Easy CI/CD Integration**
   Integrates smoothly with Jenkins, GitHub Actions, GitLab CI, etc.
8. **Scalable & Modular**
   Reuse infrastructure using **modules** as your environment grows.
9. **Cost & Resource Control**
   Plan before apply → know exactly what will be created or destroyed.
10. **Industry Standard**
    Widely used in DevOps and cloud roles; strong interview and production relevance.

🧠 One-line interview answer

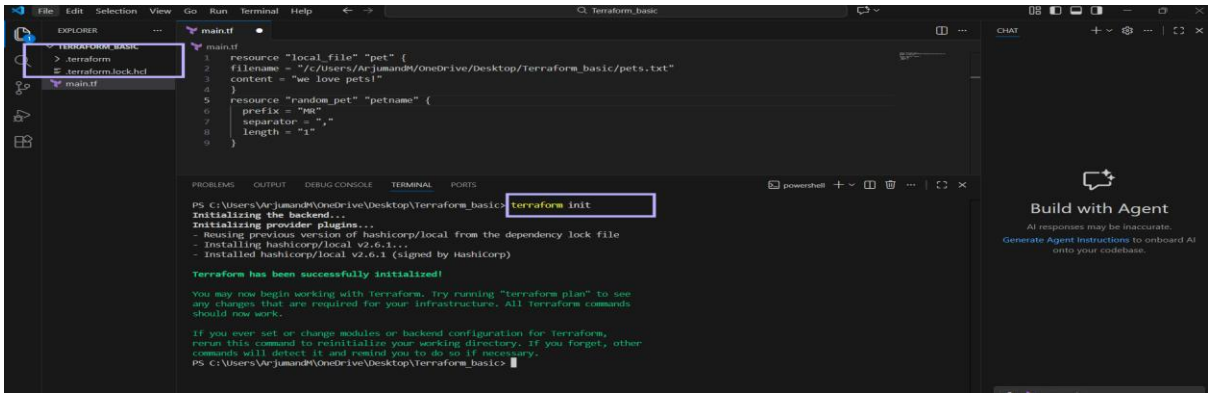
Terraform is important because it automates infrastructure provisioning using code, ensuring consistency, scalability, and multi-cloud support with safe, repeatable deployments.

Execute all the templates shown in video.

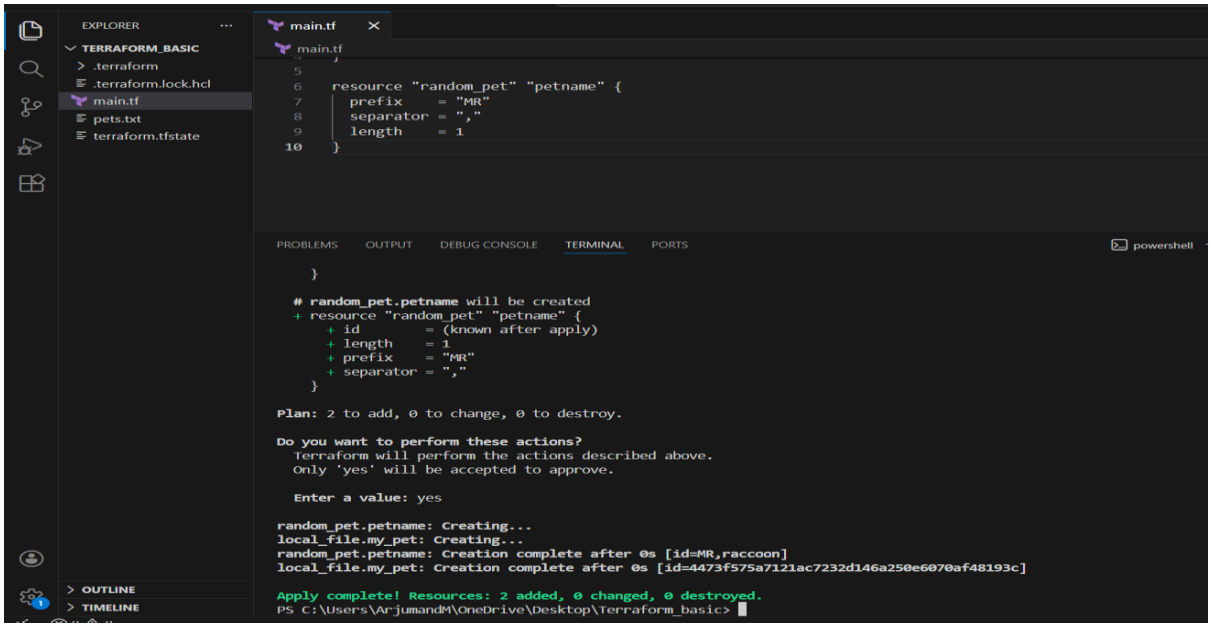
1. Note down below points, Terraform Init Terraform Plan Terraform Apply Terraform Provider.

- Open visual studio and open main.tf file which we created in last task
- As we see we are in main.tf file and destroyed previous file

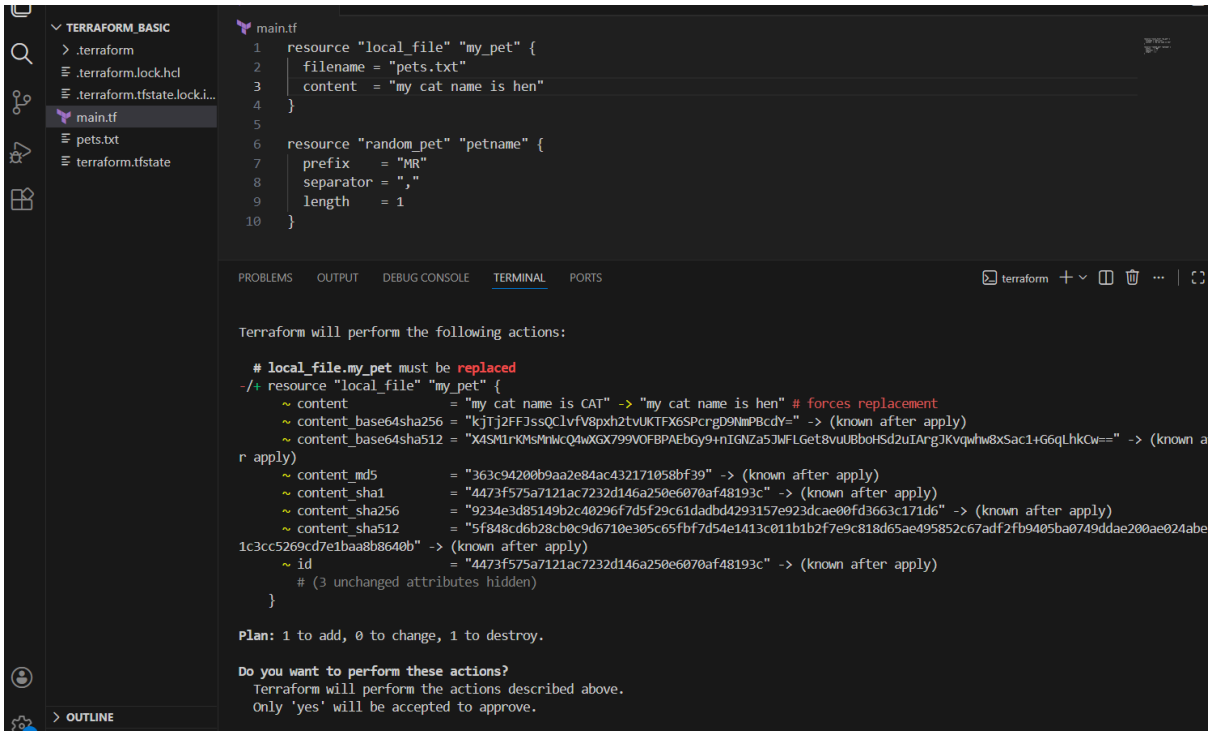
- Type terraform init it will initiate and create a backup to .terraform file as we can see



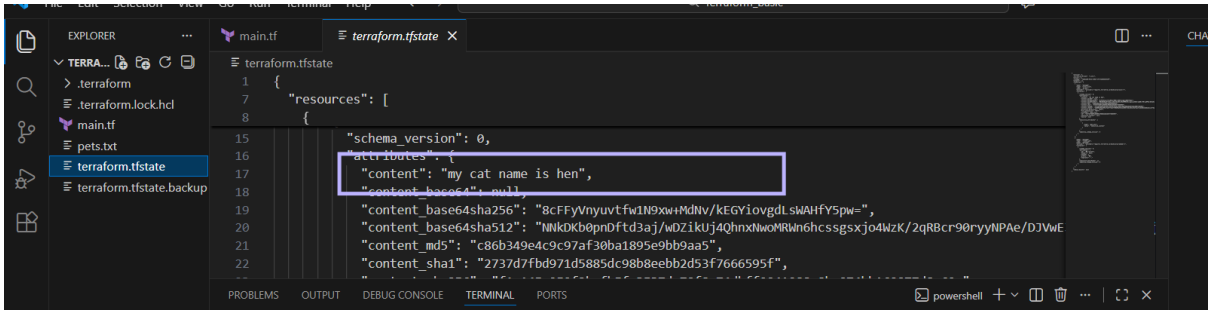
- After passing terraform apply command it will create terraform.tfstate file, it is blue print, it contain the the medata.



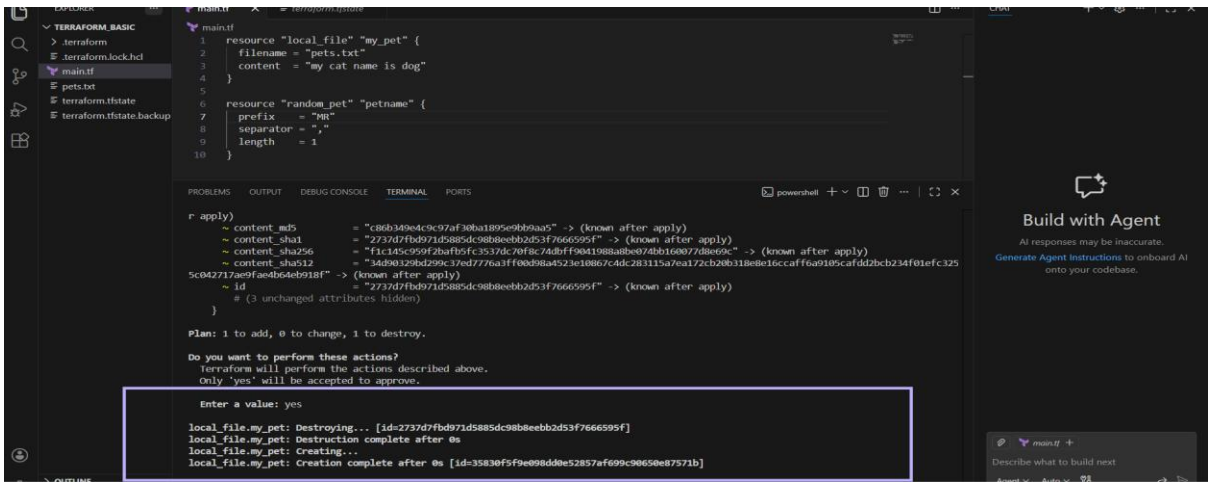
- When we make changes hen instead of cat, it will show the changes because all the records will be stored in terraform.tfstate file, from that file terraform will know the changes



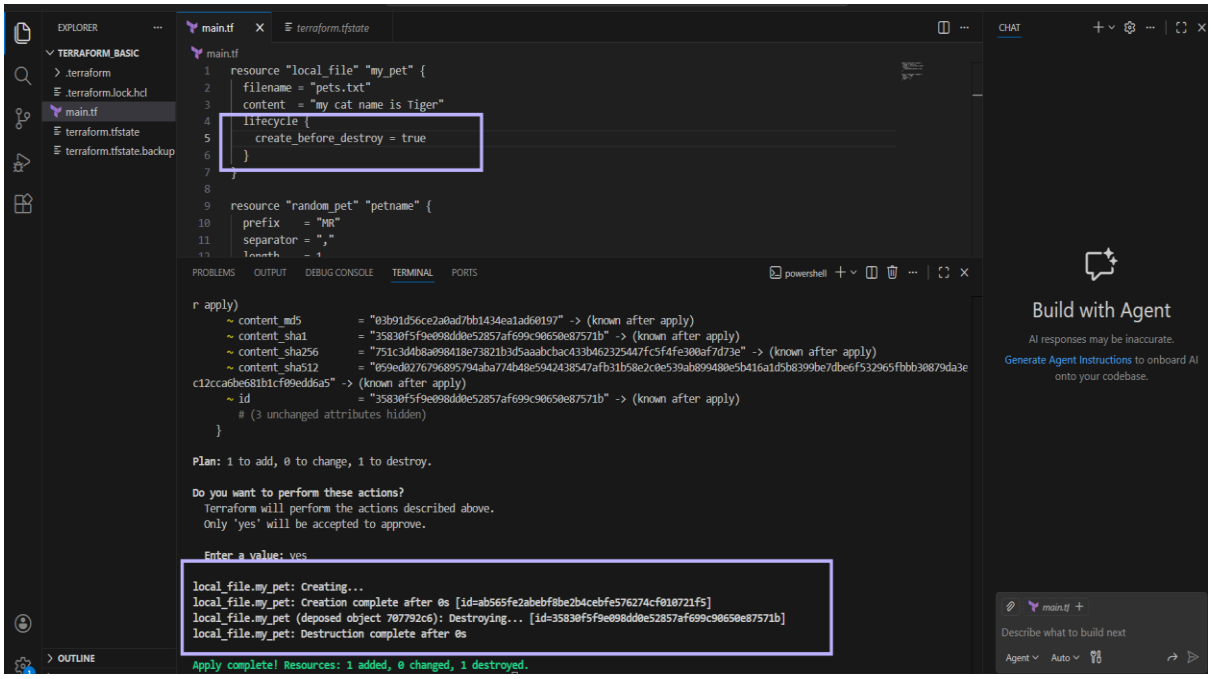
- If we go to terraform.tfstate file we can see the records along with changes



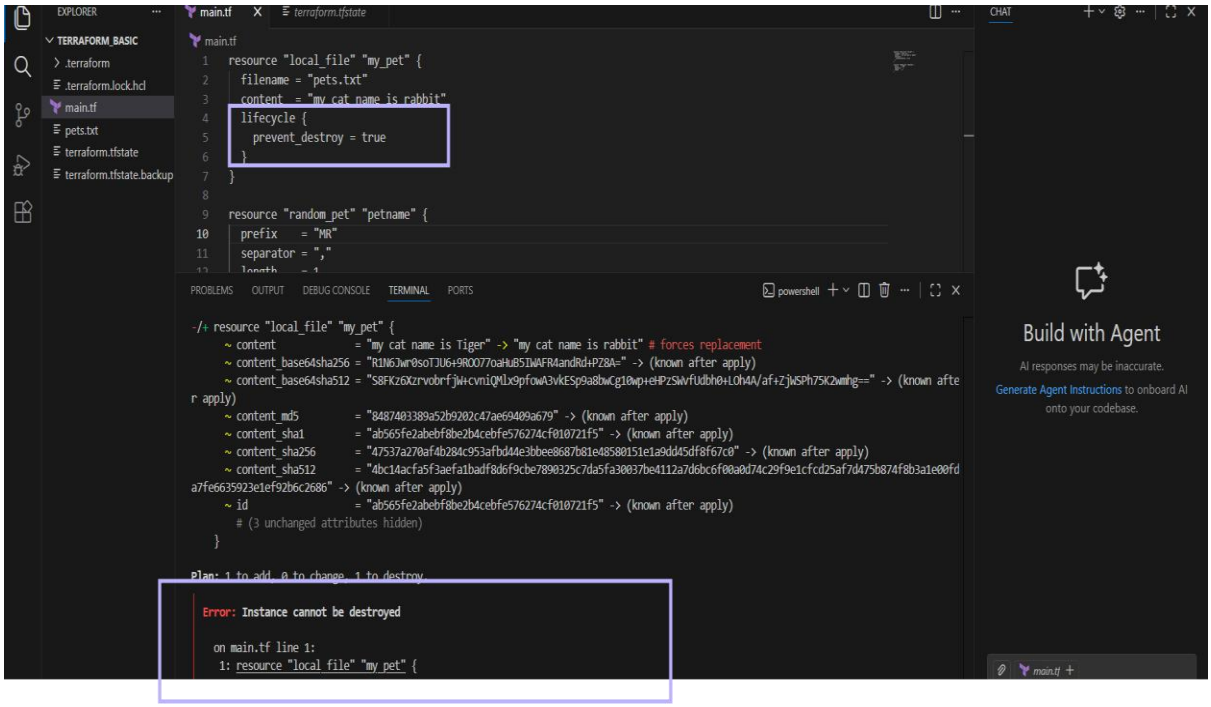
- If we are making changes by default it will destroy and then create as we can see.



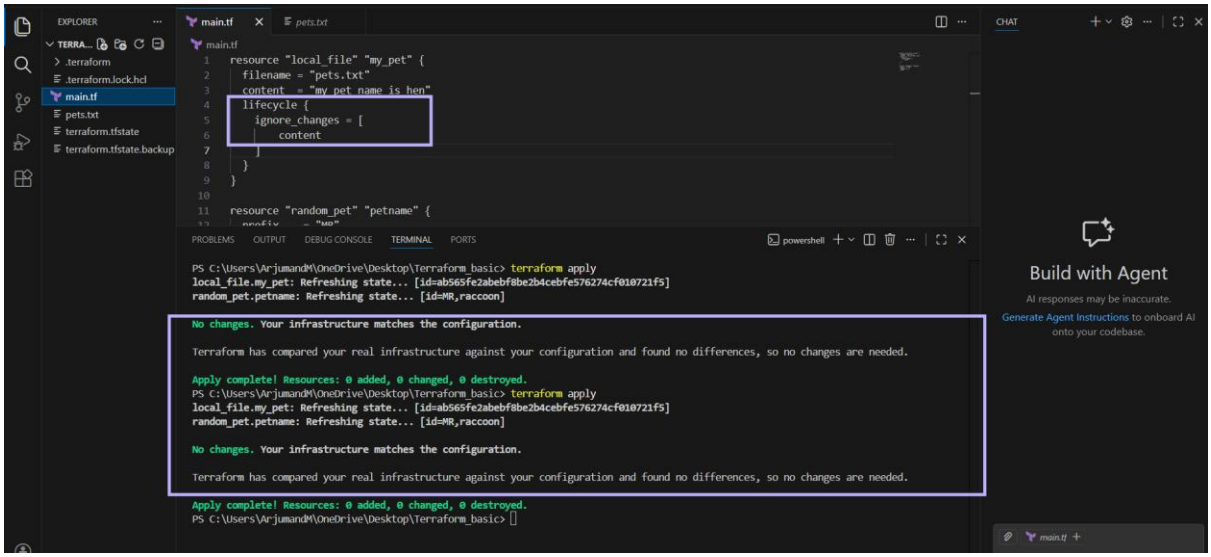
- By using lifecycle rule we can change that default changes we can ask change the role and add rule as create first before destroy,
- Following line we can add in function, lifecycle { \create_before_destroy =true \ }
- After adding that line save the file and then pass command ‘terraform apply’



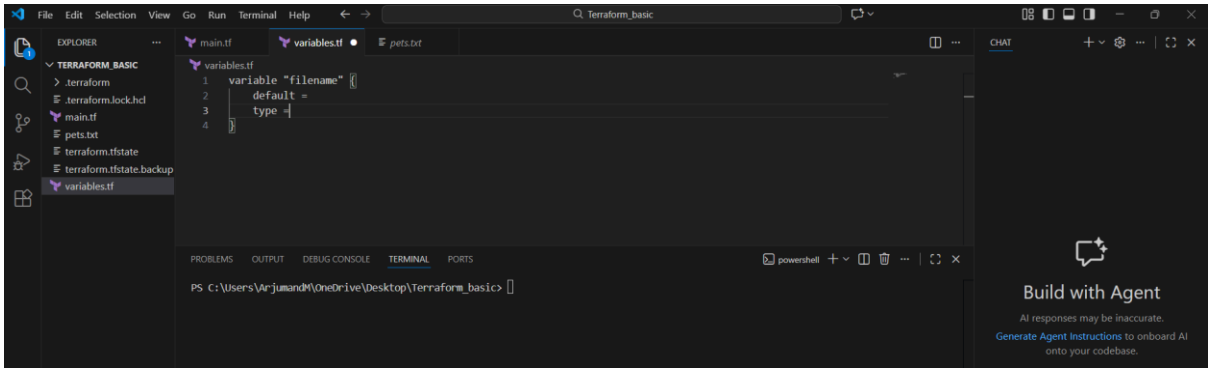
- We can also prevent from destroy as we can see in the below
- I have used following command ‘lifecycle { \ prevent_destroy = true \ }’



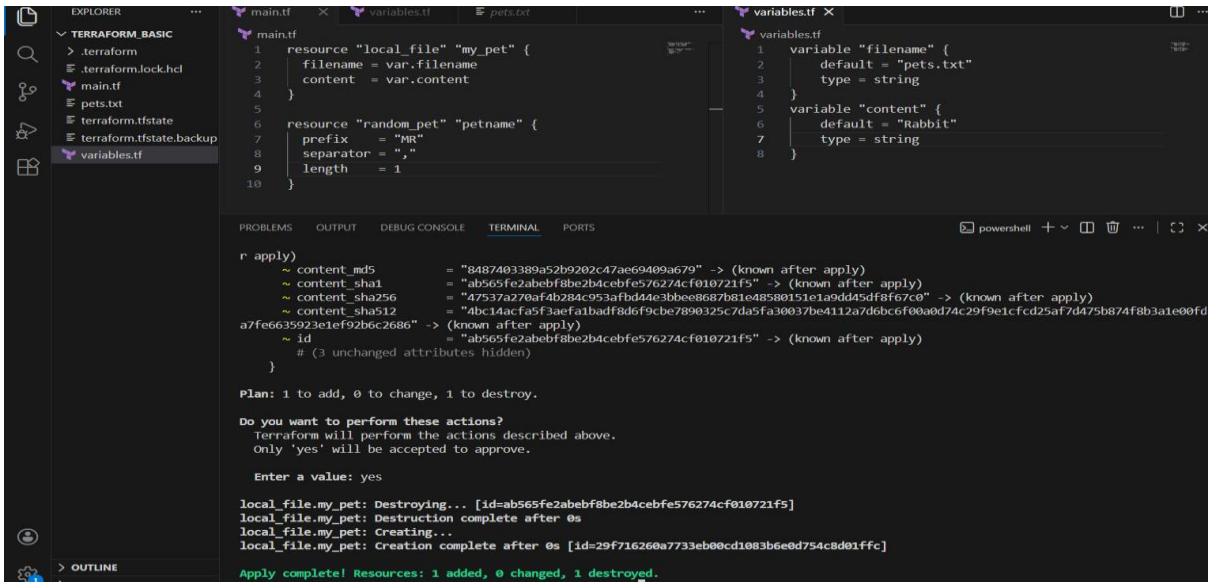
- We can also add ignore_changes as we can see in below image



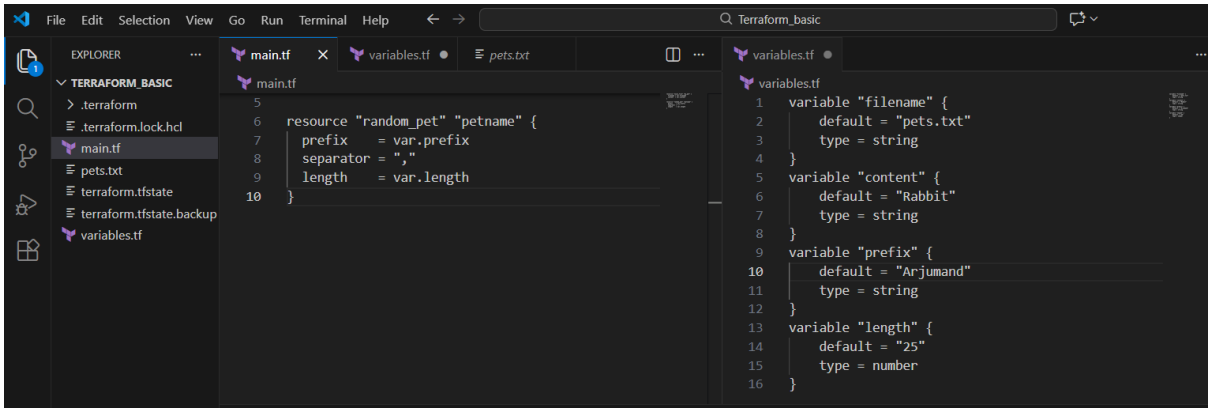
- we can variable to our file to avoid hardcoding
- Go to top and create a new file and then add variable file with .tf extension



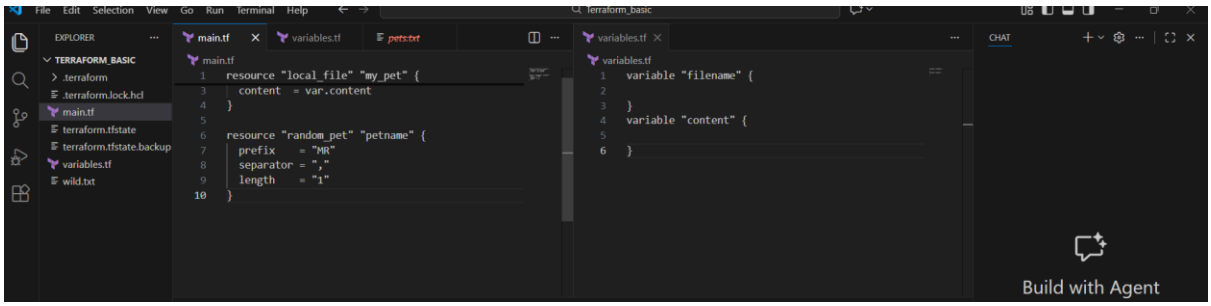
- As we can see we have added variable.tf in that we have provided variable



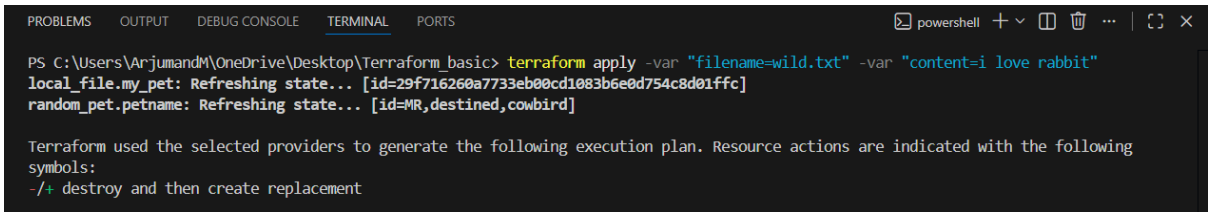
- We have added variables for all 4 resources filename, content, prefix, length



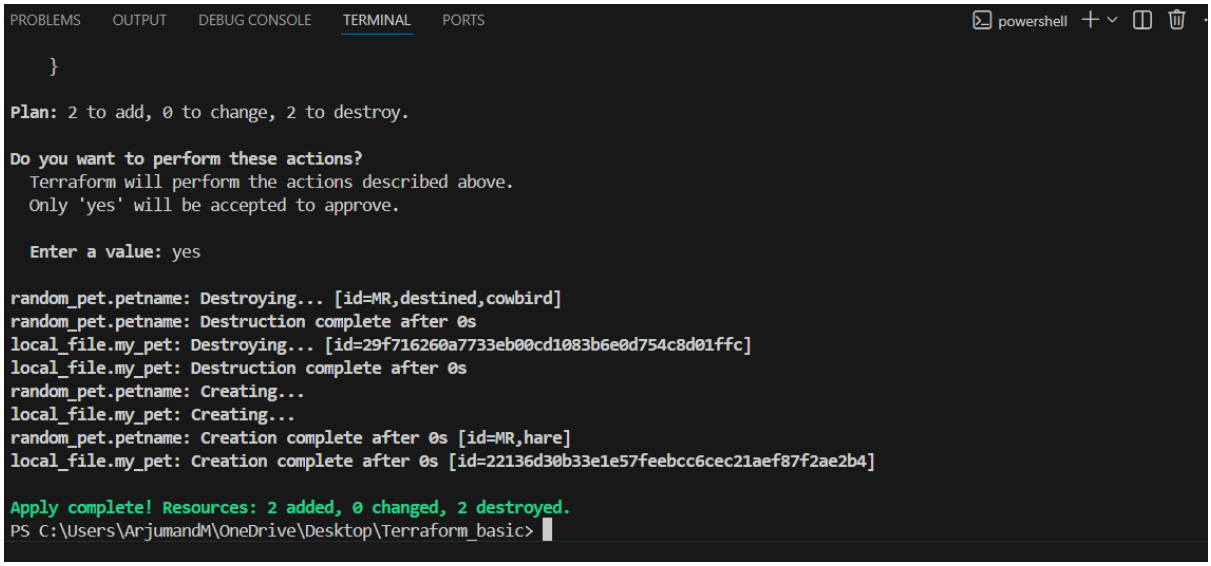
- We can also provide variable values in power cell



- I have provided variable in the PowerShell



- Variable is added sucessfully



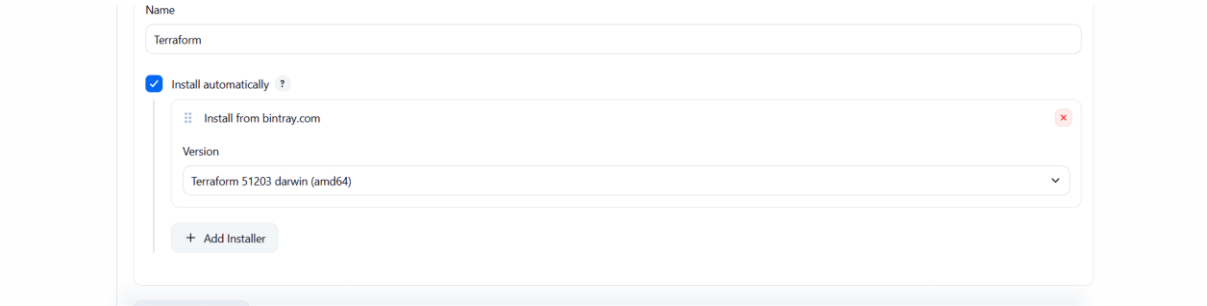
```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> terraform apply
random_pet.petname: Refreshing state... [id=MR,raccoon]
local_file.my_pet: Refreshing state... [id=29f716260a7733eb00cd1083b6e0d754c8d01ffc]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the followin
g
symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # random_pet.petname must be replaced
-/+ resource "random_pet" "petname" {
      ~ id        = "MR,raccoon" -> (known after apply)
      ~ length    = 1 -> 2 # forces replacement
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

      ~ id        = "MR,raccoon" -> (known after apply)
      ~ length    = 1 -> 2 # forces replacement
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.


Plan: 1 to add, 0 to change, 1 to destroy.


Do you want to perform these actions?
  Terraform will perform the actions described above.
```

Ln 10, C

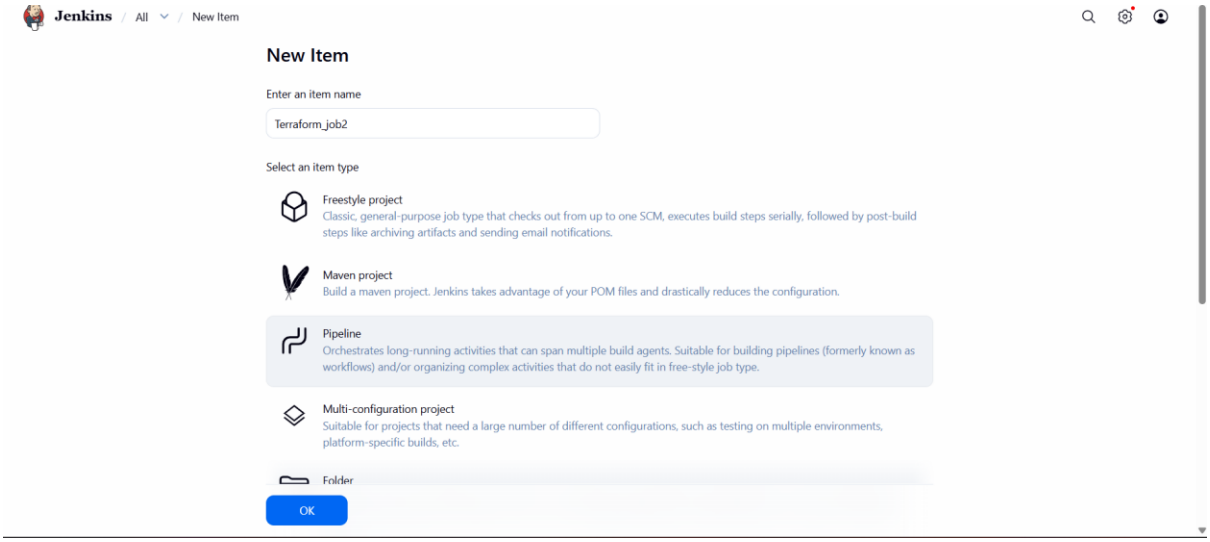
2. Integrate a sample Terraform template in Jenkins.

1. Install **Terraform Plugin** from *Manage Jenkins → Manage Plugins*.
2. Install **Terraform CLI** on the Jenkins server.
3. Verify Terraform using `terraform version`.
4. Go to *Manage Jenkins → Global Tool Configuration*.
5. Add **Terraform** tool and set Terraform binary path.
6. Create a **Pipeline job** in Jenkins.
7. Connect the job to your **Terraform Git repository**.
8. Add stages for `terraform init`, `validate`, `plan`, and `apply`.
9. Use `sh` (Linux) or `bat` (Windows) to run Terraform commands.
10. Save the job and **build the pipeline** to execute Terraform.

- After Installing the terraform plugins, go to tools and terraform



Create a new job and select pipeline and create OK




```
pipeline {
    agent any

    stages {

        stage('Checkout SCM') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/arjumand9794/terraform.git'
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }
```

```
        stage('Terraform Validate') {
            steps {
                sh 'terraform validate'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
}
```

Create a pipeline in the script

Pipeline was created successfully with my repo which has a file .tf file in it